MCP Server for HPE Aruba Networking Central

Bridging 3 000+ APIs with AI

Till Belz HP Networking Days 2026

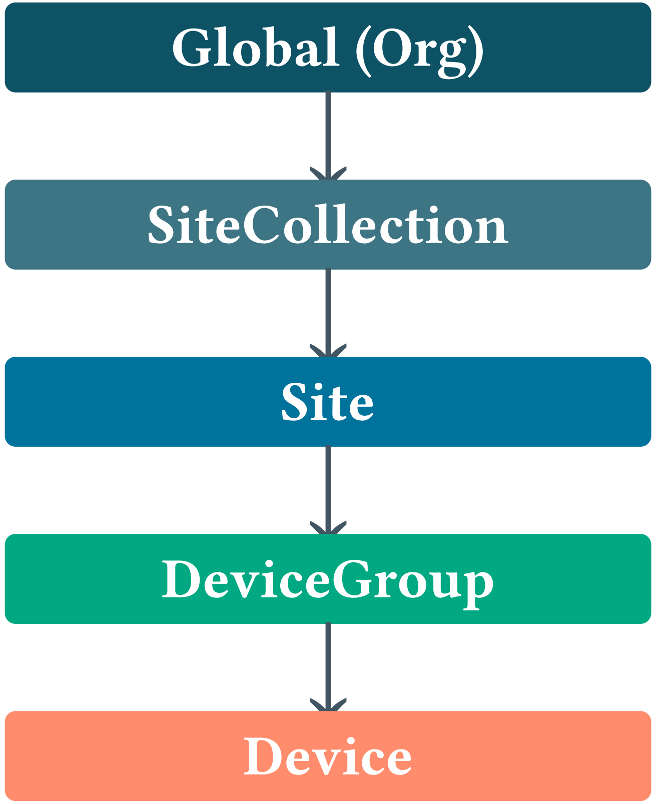
Context

Aruba Central — Configuration Model

Five-level configuration hierarchy

Config propagates **top-down**.
Most specific scope wins.

Precedence (highest → lowest):



Device › DeviceGroup › Site › SiteCollection › Global

→ *Deep dive: "Die neue Aruba Central: Praxis-Check der wichtigsten Features"*

An API-First Platform

1 500 Central API endpoints
Monitoring, Configuration, Alerting, Firmware, Topology,

60 GreenLake endpoints
Device onboarding, Subscriptions, Licensing, Locations, Identity, …

40+ API categories
Full OpenAPI specs available on developer.arubanetworks.

Troubleshooting,

...

Everything is an API. If it's an API, it's AI-ready.

Design & Architecture

The Challenge

Naïve approach: 1:1 mapping

1 API endpoint = 1 MCP tool

1 500 tools × full schemas

→

Our approach: Invocation Pattern

3 generic tools + search index

millions of tokens

✗ Context window explosion

all endpoints accessible

✓ Fits any context window

all endpoints accessible

The Invocation Pattern

1. Search

Find endpoints by keyword

```
unified_search("v
```

→

2. Discover

Read full API schema

```
get_api_endpoint_(
 "GET", "/
config/..."
)
```

→

3. Execute

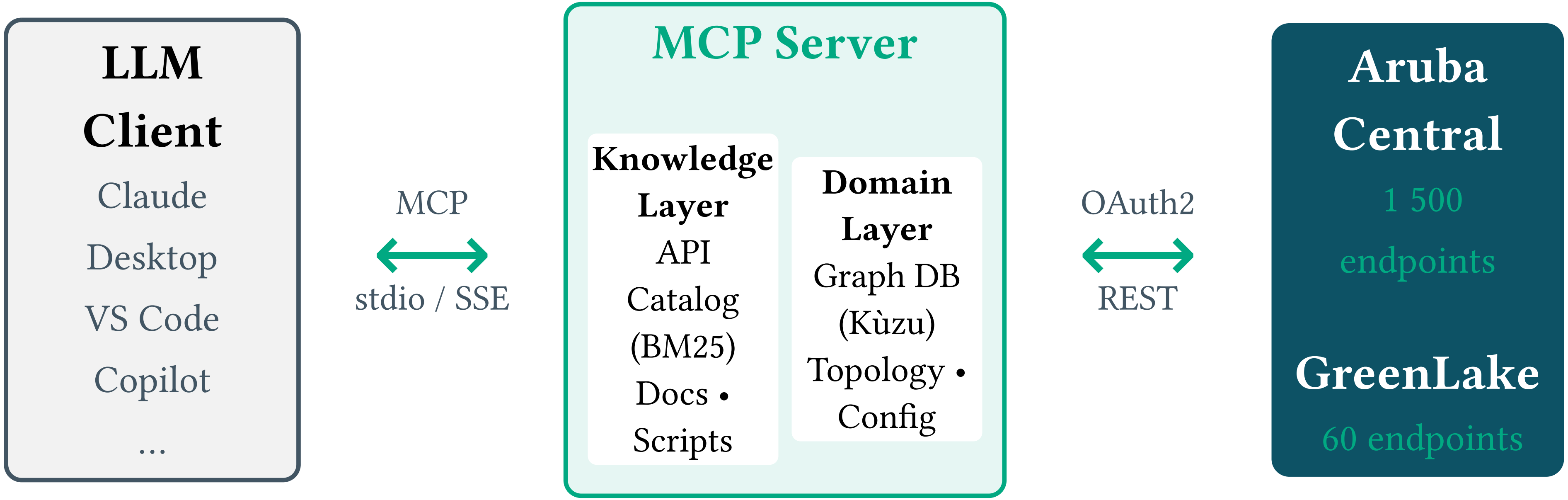
Call the API

```
call_central_api(
 "/
config/...", ...
)
```

The LLM **learns the API on-the-fly** — zero pre-loaded endpoint schemas.

all endpoints accessible

Architecture Overview



Script Engine — Sandboxed Python execution

Seeds auto-populate the graph at startup from live API data.
BM25 full-text search index over all OpenAPI specifications.

Scripts: Multi-Step Workflows

When to use scripts:

- Multiple API calls in sequence
- Pagination over large datasets
- Complex logic / conditionals
- Reusable automation

Built-in seed scripts:

- `populate_base_graph` — Org hierarchy

```
from central_helpers import
api, graph

# Pre-authenticated — no OAuth
needed
devices = api.paginate(
    "network-monitoring/
v1alpha1/devices"
)

for d in devices:
    graph.execute(
```

all endpoints accessible

- `enrich_topology` — LLDP adjacency
- `populate_monitoring` — Ports, clients
- `onboard_device` — Full onboarding flow

```
        "MERGE (d:Device
{serial: $s})"
        " ON CREATE SET d.name
= $n",
        {"s": d["serial"], "n":
d["name"]}
    )
```

The Graph Database

Embedded graph (Kùzu / LadybugDB) populated at startup:

```
Org ──→ SiteCollection ──→ Site
──→ Device

                                    ↑
         DeviceGroup ────────┘

                                     ↕

CONNECTED_TO
```

Example queries:

```
// All devices at a site
MATCH (s:Site)-
[:HAS_DEVICE]->(d:Device)
WHERE s.name = "Berlin-HQ"
RETURN d.name, d.status

// LLDP neighbors of a
switch
MATCH (d:Device)-
[c:CONNECTED_TO]->
```

all endpoints accessible

```
                        (LLDP
adjacency)
```

```
       (n:Device)
WHERE d.name = "SW-Core-01"
RETURN n.name, c.fromPorts
```

- Topology navigation via **Cypher** queries
- LLDP neighbor discovery (incl. unmanaged devices)
- Config scope tracking for blast-radius analysis
- BM25 full-text search across all node types

all endpoints accessible

11 Tools, 4 Categories

🔍 Discovery

- `unified_search` — BM25 keyword search
- `list_api_categories` — Browse API areas
- `get_api_endpoint_detail` — Full schema

⚡ Direct API

- `call_central_api` — Central REST calls
- `call_greenlake_api` — GreenLake REST calls

all endpoints accessible

Graph

- `query_graph` — Read-only Cypher queries
- `write_graph` — Enrich graph with findings

Scripts

- `list_scripts` / `get_script_content`
- `save_script` — Persist automation
- `execute_script` — Run in sandbox

Live Demos

Demo Overview

Root Cause Analysis

Find unhealthy devices,

Firmware Compliance

Check firmware versions,

⚙

VLAN Configuration

Create VLAN profile, assign to ports, verify push status

diagnose problems,
analyze topology

Demo environment

find outdated devices,
recommend upgrades

Demo environment

Lab environment

Demo 1: Root Cause Analysis

"Show me devices with problems in the network and diagnose the root causes."

Expected agent workflow:

1. Graph query

Query topology for device health

2. Alert API

Fetch active alerts and event details

3. Neighbors

Check LLDP topology for upstream issues

4. Diagnosis

Cross-correlate data and report findings

↑ *Replace this slide with a live demo screenshot or video later.*

Demo 2: Firmware Compliance

"Which devices are running outdated firmware?
Compare against recommended versions."

Expected agent workflow:

1. Search APIs

Find firmware & compliance endpoints

2. Get versions

Fetch current vs. recommended firmware

3. Compare

Cross-site comparison per device type

4. Report

Compliance summary with recommendations

↑ *Replace this slide with a live demo screenshot or video later.*

Demo 3: VLAN Configuration (Mutating)

"Create VLAN 100 on switch [name], assign it to port 1/1/5,
ensure trunk ports carry it, and verify the config was pushed."

Expected agent workflow:

1. Discover

Search config profile APIs for VLANs

2. Create

Create VLAN config profile via API

3. Assign

Assign profile to device scope

4. Verify

Check push status with effective=true

↑ *Replace this slide with a live demo screenshot or video later.*

Closing

Important

⚠ Not an official HPE product

This is a **personal / community project** that demonstrates what API-

✓ Production recommendations

- Restrict to **GET requests only**
- Full API access = full blast radius

first
platforms enable.

- Use for **monitoring & analytics**
- Be careful with mutations

Try It Out

github.com/tbelz/hpe-networking-central-mcp

Docker
One-command setup

Claude Desktop

VS Code
Copilot MCP support

Add to
config.json

Works with any MCP-compatible client • Requires Aruba Central API credentials

Key Takeaway

An API-first platform is inherently AI-ready.

Point an agent at your network —
get autonomous diagnostics, analytics, and operations.

Q & A

Questions?

Till Belz

github.com/tbelz/hpe-networking-central-mcp